

정유현

Backend Developer

전화번호 010-6231-6543

E-MAIL top6543top@naver.com

GitHub https://github.com/top6543top

INTRODUCTION

검증을 우선하는 개발자

Java·Spring 기반 웹 백엔드를 개발합니다. 금융권 폐쇄망용 자연어→SQL 서비스를 만들며, LLM이 검증 없이 만든 SQL의 40%가 없는 테이블·컬럼을 참조하는 걸 확인했습니다. 그래서 SQL은 LLM이 만들되 통과 여부는 JSqlParser와 Oracle 실행계획으로 코드가 판단하게 했습니다. 서비스에서 제일 중요한 건 안정적으로 돌아가는 코드라서, 안정성은 성능 테스트로 따로 확인합니다. MCP 기반 헥사고날 아키텍처 오픈소스를 만들 때도, 스타트업에서 OpenClaw 서비스 QC를 맡았을 때도 이 기준을 지켰습니다.

사람을 잇는 개발자

학과 학생회장과 GDGoC 팀장을 하면서 직군도 생각도 다른 사람들과 일했습니다. 계획대로 흘러가지 않는 순간이 많았습니다. 그때마다 대화를 끊지 않고 다시 맞춰 가며 일을 끝까지 가져갔습니다. 프로젝트에서도 아이디어를 먼저 꺼내고, 팀이 같은 방향을 보도록 사람 사이를 잇는 역할을 맡습니다.

EXPERIENCE

Semo

Semolabs Inc. (프리랜서) : 2025.09 ~ 2026.08
사이트 : https://semo.world

- 미들웨어 서버 Window 문제 QC
- Google Casa Auth 인증 담당
- 자동화 Agent Skill 제작

KICKBACK

Kickback (프리랜서) : 2025.10 ~ 2026.03
사이트 : https://kickbackforyou.com

- 수동 운영 프로세스를 자동화 워크플로우로 전환 기획
- Airtable·Zapier 기반 노코드 자동화 시스템 개발

대 외 활 동

동국대학교 컴퓨터AI학부 학생회 학생회장 : 2026.03 ~ 현재
동국대학교 Coss 사업단 IoT 서포터즈 3기 : 2026.04 ~ 현재
HI-SW 봉사단 : 2025.06 ~ 2025.12
Google Developer Group on Campus DGU Community Core : 2024.09 ~ 2026.06
중앙동아리 부회장 : 2022.01~2022.12
공과대학 학생회 총무부원 : 2022.01~2022.12

REPRESENTATIVE AWARD

[창의적 문제 해결 프로젝트 최우수상] FabQuest

- 반도체 8대 공정 교육 플랫폼을 개발해 실제 교육을 진행한 점을 인정 받아 최우수상 수상

[동국대학교 산학연지협력장학(창업분야)] TIGO

- 관광데이터경진대회 출품작으로 사업자 등록 후 서비스의 우수성을 인정받아 장학금 수여

[AISW 중심대학 AI 블록체인 비지니스 모델 경진대회 특별상] SEMO

- OpenClaw기반 이커머스 AI Agent인 Semo의 비지니스 모델을 인정 받아 특별상 수상

CERTIFICATE

SQLD(SQL개발자)
2024.06.21 / SQLD - 05009279
ADSP(데이터분석 준전문가)
2024.03.22 / ADsP-040006145
정보처리기사
2026.06.12 / 26201020478G
한국사검정능력시험 2급
2023.11.03 / 67-209300

P R O J E C T

[개인 프로젝트] 금융 시스템 개발 프로젝트

일정 : 2026.04 ~ 진행중

기술 스택 : Java 17, Spring Boot 3.4.1, MySQL 8.0, Redis 7, Spring Batch, AWS, k3s, Prometheus/Grafana, Testcontainers

참여 인력 : Backend 1명

서비스 : 계좌 개설·입출금·이체를 처리하는 금융 원장 API 서비스

고려사항 : 동시 요청과 재시도가 몰려도 잔액이 틀어지지 않고, 배포 중에도 거래가 끊기지 않아야 함

1. **주요 수행 업무**

   동시성 문제 추적·해결: 100건 동시 출금에서 나온 데드락의 원인을 JPA 실행 시점 차이까지 추적해 없앴고, 낙관적 락과 비관적 락을 같은 시나리오로 실측해 비교했습니다.

   멱등한 이체 API: Idempotency-Key 선점부터 응답 저장까지 한 트랜잭션에 묶어, 클라이언트가 재시도해도 돈은 한 번만 움직이게 했습니다.

   잔고 대사 배치: Spring Batch로 매일 잔액과 원장을 대조하고, 불일치는 증거 테이블에 보존해 Prometheus 경보로 이어지게 했습니다. 보안 경계: Redis 토큰 버킷으로 요청 수를 제한하고 JWT로 인증했습니다. 오류 로그에는 비밀번호나 계좌번호 같은 민감값이 남지 않게 걸렀습니다.

   CI/CD와 k3s 배포: CI를 통과한 커밋만 SHA 이미지로 빌드해 ECR digest로 고정하고, self-hosted 러너가 k3s에 Flyway 마이그레이션과 배포를 적용합니다.

2. **주요 성과**

   Testcontainers의 실제 MySQL·Redis 위에서 테스트 134개를 통과했습니다. 데드락은 4회 반복 측정 모두 0건이었고, 같은 키로 50개 요청을 동시에 보내도 원장에는 한 번만 기록되는 것을 확인했습니다.

[산학협력 프로젝트] aTopsNL2SQL 금융권 자연어→SQL 변환 및 검증 서비스

일정 : 2026.03 ~ 진행중

기술 스택 : Spring Boot 2.7.18, PostgreSQL+pgvector, JSqlParser, Oracle JDBC, Ollama/GPT

담당 : Backend Developer

서비스 : 금융권 내부 폐쇄망에 들어갈 자연어 → SQL 변환 및 검증 서비스

고려사항 : 폐쇄망이라 외부 가드레일 API·외부 AI Agent를 쓸 수 없어 검증·품질측정을 전부 자체 구현해야 했고, 수만 개의 테이블·컬럼으로 구성된 금융권 메타데이터에서 LLM 생성 SQL만으로는 정확도를 담보할 수 없었음

1. **주요 수행 업무**

   5단계 SQL 검증 하네스: 금지키워드·다중문 차단부터 SELECT/WITH-only, JSqlParser 기반 테이블·컬럼 존재성, Oracle EXPLAIN까지 저렴한 검사부터 비싼 검사 순으로 체이닝해 앞 단계 실패 시 즉시 반환하도록 설계했습니다.

   실패 유형별 재시도 루프: 검증·파싱 실패는 실패 사유를 refine 프롬프트에 재주입해 바로 재시도하고, API 예외만 지수 백오프를 적용했습니다. 최대 3회를 넘기면 미검증 SQL은 반환하지 않습니다.

   조인 신뢰도 산출 배치: FK 제약이 0건인 Oracle 스키마에서 PK 매칭·데이터타입 일치·최소 등장빈도 3중 조건으로 조인쌍을 확정하는 PkVerificationBatch를 만들었습니다.

   조인 후보 랭킹: 빈도 × 최신성(180일 반감기) × PK 확정 보너스로 가중치를 계산하고, 복합 인덱스로 정렬된 후보를 바로 가져오게 했습니다.

2. **주요 성과**

   골든 15건 실측에서 하네스 없이 1회 생성하면 40%가 존재하지 않는 테이블/컬럼을 참조했고, 하네스를 거친 뒤 최종 차단은 26.7%(4건), 미검증 SQL 노출은 0건이었습니다. 통과 케이스만 비교하면 정밀도가 raw 대비 평균 3%p 개선됐습니다. 검증·파서·배치 로직은 JUnit 테스트 18건으로 회귀 검증합니다.

PORTFOLIO

금융 시스템 개발 프로젝트

동시 출금·재시도·트래픽 제한에서 돈이 틀어지던 지점과 해결 과정

IDENTITY 전략이 만든 동시 출금 데드락

문제 계좌 1개에 100건을 동시에 출금하자 성공 12건, 실패 88건이 나왔습니다. 로그를 보니 실패 대부분은 버전 충돌이 아니라 MySQL 데드락이었습니다.

고려 처음엔 보조 인덱스 락을 의심했지만, 일반 SELECT는 락을 걸지 않고 UPDATE는 항상 PK로 나가 원인이 아니었습니다. 진짜 원인은 IDENTITY 때문에 이력 INSERT가 먼저 나가 FK 체크로 S-lock을 잡고, 계좌 UPDATE는 더티체킹으로 커밋 시점에야 X-lock을 요청하는 순서 역전이었습니다.

변경 전: 이력 INSERT가 먼저 나가 S-lock끼리 X-lock을 막음

	T1 (출금 A)	계좌 #1 행 락	T2 (출금 B)
1	이력 save → IDENTITY라 즉시 INSERT, FK 체크	**S: T1**	
2		**S: T1, T2**	이력 INSERT, FK 체크
3	커밋 → 더티체킹 UPDATE, X 요청	**T1 대기 (T2의 S)**	
4		**T2 대기 (T1의 S)**	커밋 → UPDATE, X 요청

서로의 S-lock을 기다리는 순환 대기. MySQL이 한쪽을 데드락으로 롤백 (100건 중 성공 12, 실패 88)

변경 후: 계좌 UPDATE를 먼저 flush해 X-lock부터 잡음

	T1 (출금 A)	계좌 #1 행 락	T2 (출금 B)
1	잔액 변경 후 flush() → UPDATE	**X: T1**	
2		**T2 대기 (T1의 X)**	flush() → UPDATE, X 요청
3	이력 INSERT → 커밋	**해제**	
4		**X: T2**	version 불일치 → @Retryable 재시도

락 요청이 X-lock 하나로 줄 서서 순환이 생기지 않음 (4회 측정 모두 데드락 0건)

조치 이력 저장 전에 flush로 계좌 UPDATE를 먼저 내보내 X-lock부터 잡도록 순서를 바꿨습니다.

결과 4회 반복 측정 모두 데드락 0건이었습니다. 남은 실패는 순수한 버전 충돌이라 @Retryable로 흡수했습니다.

응답 유실 뒤 재시도에도 한 번만 반영되는 이체

문제 응답이 유실된 뒤 들어오는 재시도는 새 요청처럼 보여서 낙관적 락으로 막을 수 없었습니다.

고려 애플리케이션에서 조회한 뒤 삽입하는 방식으로는 동시 요청 사이에 빈틈이 남아서 DB 제약을 쓰기로 했습니다.

조치 사용자+Idempotency-Key에 UNIQUE 제약을 걸고, 키 선점과 잔액 변경, 원장 기록, 응답 저장을 한 트랜잭션으로 묶었습니다.

결과 같은 키로 50개 요청을 동시에 보내도 원장에는 한 번만 기록되는 것을 확인했습니다.

위조 헤더와 Redis 장애로 뚫리던 Rate Limit

문제 처음에는 X-Forwarded-For 헤더로 IP를 식별하고 Redis 장애 시 요청을 허용하도록 설계했는데, 헤더를 위조하거나 장애 상황을 이용하면 제한을 우회할 수 있었습니다.

고려 다중 Pod가 같은 한도를 공유하면서 동시 요청에도 토큰이 이중으로 쓰이지 않아야 했고, 로그인은 일반 API보다 엄격하게 제한해야 했습니다.

	처음 설계	변경 후
식별 키	X-Forwarded-For 헤더. 값을 바꿀 때마다 새 버킷이 생김	연결 remoteAddr. 헤더를 위조해도 같은 버킷
토큰 차감	조회와 차감이 분리돼 여러 Pod가 같은 잔량을 읽고 통과	Lua 스크립트 하나로 조회와 차감을 원자적으로 처리
Redis 장애	요청 허용	503으로 거절
로그인	일반 API와 같은 버킷	로그인 전용 버킷, 60초 10회

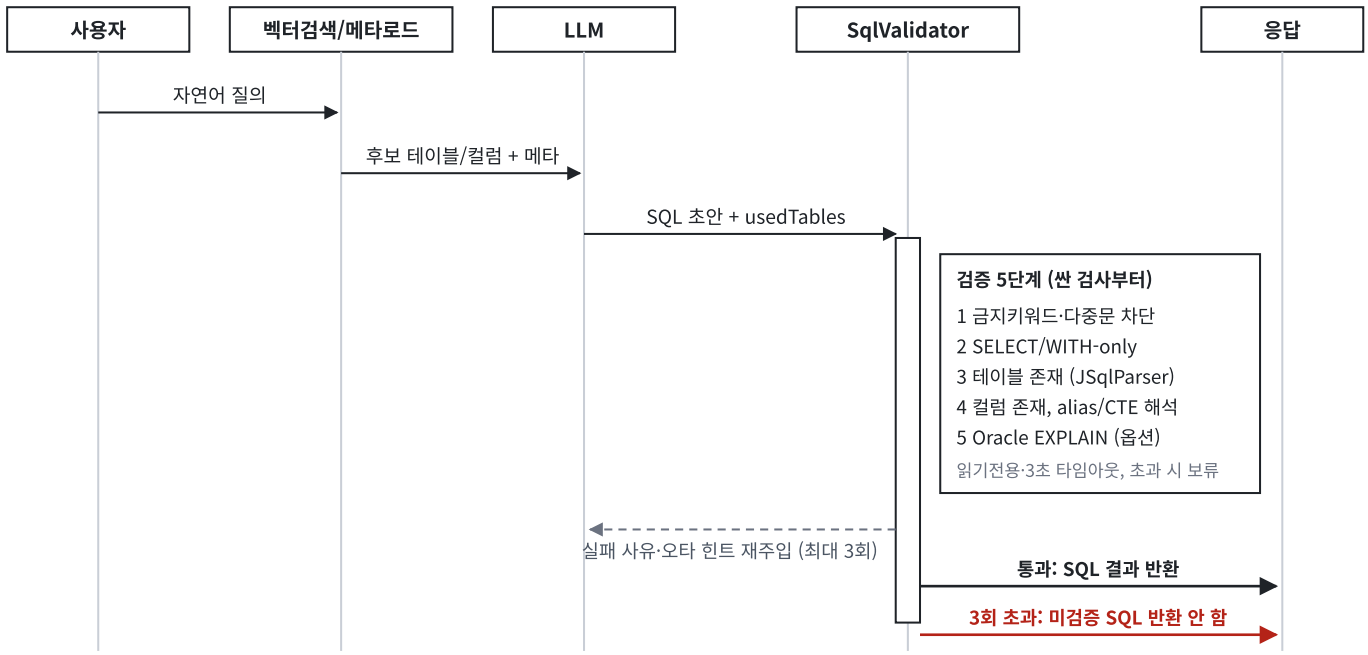
조치 조회와 차감을 Lua 스크립트 하나로 원자적으로 처리하고, 로그인 버킷은 60초 10회로 분리했습니다. 식별 기준은 연결 remoteAddr로 바꾸고, 장애 시에는 503으로 거절하게 했습니다.

결과 위조 헤더로는 제한 키가 바뀌지 않았고, 실제 Redis에서 동시 요청 50개 중 정확히 5개만 허용되는 것을 확인했습니다.

PORTFOLIO

aTopsNL2SQL · 금융권 자연어→SQL 변환 및 검증 서비스

산학협력 프로젝트. 폐쇄망에서 LLM이 만든 SQL을 코드로 검증하는 구조



자연어 → SQL 생성과 검증 순서. 검증·파서·배치 로직은 JUnit 테스트 18건(SqlValidatorTest, ColumnHintGeneratorTest, OracleExplainValidatorIntegrationTest 등)으로 회귀 검증합니다.

비용 순서로 정렬한 5단계 SQL 검증 하네스

문제 폐쇄망 환경이라 외부 가드레일 API를 호출할 수 없었고, LLM 생성 SQL을 문자열 매칭으로만 검증하면 alias·서브쿼리·CTE가 섞인 실제 쿼리에서 오탐·누락이 잦았습니다.

고려 정규식 키워드 필터만 두면 컬럼 오타·존재하지 않는 테이블을 못 잡았고, 모든 SQL에 Oracle EXPLAIN을 먼저 돌리면 대형 테이블·다중 조인에서 응답이 느려질 위험이 있었습니다.

조치 금지키워드·다중문 차단 → SELECT/WITH-only → JSqlParser 기반 테이블 존재성 → alias/CTE 해석 컬럼 존재성 → (옵션)Oracle EXPLAIN 순으로, 저렴한 검사부터 비싼 검사 순으로 체이닝해 앞 단계 실패 시 즉시 반환하도록 설계했습니다. EXPLAIN은 읽기전용 계정 + 3초 타임아웃입니다.

결과 실제 SQL 파서·Oracle 실행계획 기반 검증으로 정확도를 확보했습니다. 골든 15건 실측에서 하네스 없이 1회 생성하면 40%가 존재하지 않는 테이블/컬럼을 참조했고, 하네스를 거친 뒤 최종 차단은 26.7%(4건), 미검증 SQL 노출은 0건이었습니다. 통과 케이스만 비교하면 테이블 선택 정밀도가 raw 대비 평균 3%p 개선됐고, 재현율은 동일했습니다.

조건	값	기준
검증 없이 1회 생성 (RAW)	테이블 재현율 30.1%	RoundTripEvalMain
검증 + 최대 3회 재시도 (HARNESS)	테이블 재현율 27.9%	차단 케이스를 0점으로 포함한 평균
통과 케이스만 비교	정밀도 +3%p	N=11 (차단 4건 제외), 재현율은 동일
3회 재시도 후에도 실패	최종 차단 26.7%	4/15건, 미검증 SQL 노출 0건

하네스 재현율이 낮아 보이는 건 차단된 4건을 SQL 없이 0점 처리했기 때문입니다. 하네스는 재현율을 올리는 장치가 아니라 확신 없는 SQL을 내보내지 않는 안전장치입니다.

실패 유형을 구분한 재시도 루프

문제 SQL 생성 실패에는 "검증 규칙 위반"처럼 즉시 다시 물어야 하는 경우와, "API 레이트리밋/네트워크 오류"처럼 즉시 재시도하면 오히려 악화되는 경우가 섞여 있었습니다.

고려 모든 실패에 동일한 대기시간을 주는 단순 재시도가 구현은 가장 쉬웠지만, 검증 실패까지 매번 기다리면 응답이 느려지고 API 오류까지 즉시 재시도하면 레이트리밋을 더 악화시킬 수 있었습니다.

조치 SqlGenAiService.run()에서 검증/파싱 실패는 GenRetryException으로 묶어 실패 사유를 refine 프롬프트에 재주입해 바로 재시도하고, 그 외 API 예외만 시도 횟수에 비례한 지수 백오프를 적용했습니다. 최대 3회를 넘기면 미검증 SQL은 반환하지 않습니다.

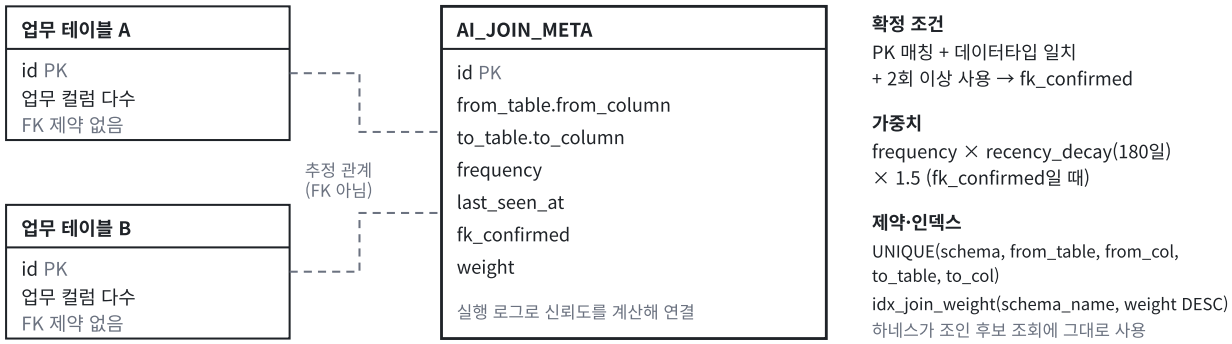
결과 검증 실패 경로는 지연 없이 재시도되고, 일시적 API 장애 경로만 점증 대기하도록 분리해 외부 API에 부담을 주지 않는 구조를 만들었습니다.

FK 없는 레거시 스키마에서 조인 신뢰도 직접 산출

문제 ATOPS 스키마는 Oracle FK 제약조건이 0건이었습니다. 대량 배치·정산 처리 시 체크 오버헤드 때문에 설계 단계부터 FK를 걸지 않은 것으로 보이며, LLM이 아무 컬럼이나 조인에 쓰면 실행은 되지만 결과가 틀어지는 조인이 섞여 있었습니다.

고려 PK 매칭 여부만으로 조인을 확정하면 이름이 같은 PK 컬럼과 매치되는 오탐 위험이 있었고, 매 요청마다 Oracle 딕셔너리를 조회하면 폐쇄망에서 요청 지연이 커졌습니다.

조치 PkVerificationBatch가 Oracle 딕셔너리를 읽기전용 배치로 조회해, PK 매칭 + 양쪽 컬럼 데이터타입 일치 + 최소 등장빈도(2회 이상) 3중 조건을 통과한 조인쌍만 fk_confirmed로 확정했습니다. 조인 후보는 weight = frequency × recency_decay(180일 반감기) × fk_confirmed 보너스(1.5배)로 정렬하고, idx_join_weight(schema_name, weight DESC) 복합 인덱스를 함께 설계했습니다.



결과 FK 선언이 전혀 없는 레거시 스키마에서도 요청 경로에서는 매번 Oracle을 조회하지 않고 사전에 검증된 조인 신뢰도를 코드로 확보했습니다. 조인 후보는 전체 스캔 없이 인덱스를 탄 정렬로 즉시 가져오고, 계산 로직은 별도 단위 테스트로 검증했습니다.